

LESSON 1

Summary Values

Data Aggregation Fundamentals

- A **data summary** provides insights that would otherwise be difficult to recognize by examining raw data.
- **Data aggregation** is the data manipulation operation used to summarize data by applying operations to specific columns to produce single values.

Design Framework

- To effectively design a data aggregation operation, answer two key questions:
 1. **Which column do we wish to summarize?** Select the column containing values relevant to your question.
 2. **Which summary operation do we wish to apply?** Choose the appropriate operation based on the insight you need.

Mean vs. Median

- The **mean** is the sum of all values divided by the number of values.
- The **median** is the middle value when data is sorted.
- The median is not affected by outliers because it only considers the middle value(s), while the mean is influenced by outliers because it takes into account all values.
- Choose median when you don't want outliers to affect your summary, and mean when there are no outliers or when you want to capture their effect.

Working with DataFrames

- In Python, a table of data is called a **DataFrame**. We often create a new DataFrame by loading data into Python from CSV files.

```
orders_df = pd.read_csv('orders.csv')
```

- To extract a column from a DataFrame, use the bracket `[]` operator. The column is returned as a **Series**.

```
amounts = orders_df['amount']
```

Summarizing Data in Python

- To summarize data in Python, we apply our aggregation method on the extracted columns:

```
amounts = orders_df['amount']  
total_revenue = amounts.sum()
```

- The key summary methods:
 - `sum()` : Calculates the total of all values
 - `mean()` : Calculates the average value
 - `median()` : Identifies the middle value when data is sorted
 - `min()` and `max()` : Finds the minimum or maximum value
 - `count()` : Counts the number of rows or values
 - `nunique()` : Counts the number of unique values

LESSON 2

One Grouping Column

Grouped Aggregation

- **Grouped aggregation** calculates summary statistics for each distinct value in a grouping column, producing a summary table.
- Individual summary values tell us about the overall dataset, but grouped aggregation reveals patterns and variations across categories.
- The summary table has one row per group and one column per metric.

Design Framework

- To design an effective grouped aggregation, answer three questions:
 1. **By which column do we group?** The grouping column determines how rows are split into groups.
 2. **Which column(s) do we summarize?** The input columns on which metrics are calculated.
 3. **Which summary operation(s) do we apply?** Common operations include count, sum, mean, and unique count.
- Thinking through these questions before writing code is one of the most effective habits of successful data analysts.

Split-Apply-Combine

- Grouped aggregation works through a three-step process:
 1. **Split**: Data is divided into groups based on unique values in the grouping column.
 2. **Apply**: Summary operations are performed on each group separately.
 3. **Combine**: Results from each group are assembled into a single summary table.

Uniqueness

- `unique()` returns the unique values in a column.

```
df['column_name'].unique()
```

- Before grouping, use `unique()` to explore what groups exist in your data. This reveals unexpected categories and helps you anticipate issues before aggregation.

groupby() + agg()

- The `groupby()` method groups rows by a column.
- The `agg()` method calculates summary statistics using named aggregation.
- Each aggregate expression should be aliased for clarity.
- Call `reset_index()` to convert the grouping column back to a regular column.

```
grouped_df = df.groupby('grouping_column')

summary_df = grouped_df.agg(
    total_count=('column_1', 'count'),
    total_amount=('column_2', 'sum')
)

summary_df = summary_df.reset_index()
```

- Or, as a single chained expression:

```
summary_df = df.groupby('grouping_column').agg(
    total_count=('column_1', 'count'),
```

Frequency Tables

- Use `value_counts()` for quick frequency analysis when you only need counts.
- Add `normalize=True` to get proportions instead of counts.

```
df['column_name'].value_counts()  
df['column_name'].value_counts(normalize=True)
```

Sorting

- The `sort_values()` method reorders rows by a specified column.
- By default, sorting is ascending (lowest to highest). Use `ascending=False` for descending order.

```
summary_df = summary_df.sort_values(by='total_amount', ascending=False)
```

LESSON 3

Multiple Grouping Columns

Grouping by Multiple Columns

Pass column names as a list to `groupby()` to group by multiple columns. Each combination of values forms a distinct group.

```
summary_df = df.groupby(['grouping_col1', 'grouping_col2']).agg(  
    metric_1=('column_1', 'count'),  
    metric_2=('column_2', 'nunique'),  
    metric_3=('column_3', 'sum'),  
    metric_4=('column_3', 'mean')  
) .reset_index()
```


Multi-line Method Chaining

Use parentheses to break method chains across multiple lines for readability.

```
summary_df = (  
    df  
    .groupby(['grouping_col1', 'grouping_col2'])  
    .agg(  
        metric_1=('column_1', 'count'),  
        metric_2=('column_2', 'nunique'),  
        metric_3=('column_3', 'sum'),  
        metric_4=('column_3', 'mean'))  
    .reset_index()  
)
```

Group Size Considerations

- Always include the group size (typically a count) in summary tables to evaluate which summary values can be trusted.
- Summaries based on small groups are often unreliable. As a general guideline, aim for group sizes of 30 or higher.

Trade-offs of Multiple Grouping Columns

- You can group by as many columns as needed for the desired level of detail.
- Each additional grouping column multiplies the number of resulting groups.
- Too many grouping columns can lead to:
 - Overwhelming number of summary values difficult to interpret
 - Smaller group sizes producing less reliable statistics
 - Difficulty extracting meaningful insights

Column Order Importance

- The order of columns in `groupby()` determines the organization of results.
- The first column determines the primary grouping.

Handling Missing Groups

- Groups with no data will not appear in the summary table.
- Missing groups can occur for three reasons:
 1. **Data Loss or Query Error:** Data was lost, incompletely collected, or a bug in the code failed to capture existing combinations—something went wrong that needs investigation.
 2. **No Instances Occurred:** The combination is valid but had zero activity in the data—the data is correct.
 3. **Impossible Combination:** The combination cannot exist by definition.

Multi-Column Sorting

Pass a list to `sort_values()` for multi-column sorting. Use a list of booleans for mixed direction sorting.

```
# Same direction for all columns
df = df.sort_values(by=['grouping_col1', 'metric_1'], ascending=False)

# Mixed directions
df = df.sort_values(
    by=['grouping_col1', 'metric_1'],
    ascending=[True, False]
)
```

LESSON 4

Basic Transformations

Data Transformation

- Data Transformation is a fundamental data manipulation operation that allows us to derive new columns from existing columns.
- With data transformation, the output table is a copy of the original input table with the new columns attached at the end. This is in contrast to data aggregation, which summarizes the data, providing a higher level, less granular view.

Data Transformation Framework

We developed a framework for designing data transformation operations effectively which comprises two questions:

- Which column(s) do we need to use as input to obtain the desired column?
- What formula do we need to apply to the input column(s) to obtain the desired output?

Data Transformation in Pandas

We can create new columns in a DataFrame by assigning a value or formula to a new column name.

```
df["col_eur"] = df["col_usd"] * 0.92
```

Calculating Ratios

- Ratios measure relationships between different metrics
- Ratios are calculated by dividing one column value by another

```
df["ratio"] = (  
    df["col_a"] / df["col_b"]  
)
```

LESSON 5

Complex Transformations

Calculating Percent of Total

- Percent of total normalizes values to a common scale, making relative size immediately clear
- To calculate percent of total, divide each value by the sum of all values and multiply by 100

```
df["col_a_pot"] = (100 * df["col_a"] /  
                  df["col_a"].sum())  
  
df["col_b_pot"] = (100 * df["col_b"] /  
                  df["col_b"].sum())
```

- We can use data summary methods like `sum()`, `mean()`, `min()`, `max()`, `median()` in the transformation formula.

Rounding and Transformation Functions

- Use the `round()` function to control decimal places for readability
- Rounding is one example of many transformation functions; AI assistants can help you find the right function for your needs

```
df["col_a_pot"] = round(df["col_a_pot"], 2)
```

Complex Expressions

- Use parentheses to group operations for clarity and to control operator precedence. When in doubt, add parentheses.
- Format long expressions across multiple lines by wrapping in parentheses.

```
df["margin_pct"] = (  
    100 * (df["revenue"] - df["cost"]) /  
    df["revenue"]  
)
```

Multi-Step Transformations

- Break complex transformations into intermediate columns to make code easier to write, read, and debug.
- Name intermediate columns descriptively so the logic is self-documenting.
- Remove intermediate columns after development if not needed downstream.

Interpretation Considerations

- Negative values in percentage calculations can lead to percentages exceeding 100%.

LESSON 6

Basic Filtering

Filtering is among the most frequent data manipulation operations we apply to our data to extract only those rows that are relevant for the purpose at hand.

Basic Filtering

- To filter a DataFrame, we use the `loc[]` selector and pass it the filtering condition.

```
df.loc[df['col_1'] == "value"]
```

- We can select columns and filter rows at the same time using the `loc[]` operator.

```
df.loc[df['col_1'] == "value", ['col_2', 'col_3']]
```

Comparison Operators

- We can use any of the common comparison operators: `==`, `!=`, `<`, `>`, `<=`, `>=` to construct the filtering condition.

Partial String Matching

- To filter rows based on partial string patterns:
 - `str.contains('value')` : contains 'value' anywhere
 - `str.startswith('value')` : starts with 'value'
 - `str.endswith('value')` : ends with 'value'

```
df.loc[df['col_1'].str.contains('value')]
df.loc[df['col_1'].str.startswith('value')]
df.loc[df['col_1'].str.endswith('value')]
```

Missing Values

- To filter rows where a column is null (missing), use `isna()`.
- To filter rows where a column has a value, use `notna()`.

```
# Find rows with missing values
df.loc[df['col_1'].isna()]

# Find rows with values
df.loc[df['col_1'].notna()]
```


Selecting Columns

- By extracting the relevant columns, we can focus on the relevant information making us more efficient and potentially leading to more insightful results.
- We carry out thoughtful **column selection** at the onset of a project and also **before each sub-task in the project**.
- Effectively identifying the relevant columns requires having **clarity on the desired output** and **familiarity with the data**.
- The operation of extracting a subset of columns into a separate table is called "Selecting Columns" or just "**Selecting**".

Selecting Columns in Pandas

- We can select columns by name using `loc[]` which returns a new DataFrame with only the specified columns.

```
df.loc[:, ['col_1', 'col_2', 'col_3']]
```

- Selecting a single column using the column name returns a Pandas Series and is suitable for applying methods to a single column.

```
df['col_1']
```

LESSON 7

Multiple Conditions

AND and OR Operations

- We can combine multiple filtering conditions using `AND` and `OR` operations.
- The `AND` operation returns rows that satisfy both conditions.
- The `OR` operation returns rows that satisfy either condition.

Condition 1 Condition 2 AND Result OR Result

True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

Logical Operator Precedence

- By default, `AND` operations are executed before `OR` operations.
- We can use parentheses to enforce a particular order of execution.

Design Framework

We developed the following framework for designing filtering operations:

- 1. What filtering conditions do we need to apply?**
- 2. If applicable, how do we combine multiple conditions?**

Filtering by Multiple Conditions

- We can combine multiple filtering conditions using the `&` (AND) and `|` (OR) operators.
- The `&` operator returns rows that satisfy both conditions.

```
df.loc[(df['col_1'] == 'value_a') &  
       (df['col_2'] == 'value_b')]
```

- The `|` operator returns rows that satisfy either condition.

```
df.loc[(df['col_1'] == 'value_a') |  
       (df['col_2'] == 'value_b')]
```

- To check if a column value is in a list of values, we use the `isin()` method, which is equivalent to having multiple equality (`==`) conditions combined with OR (`|`).

```
df.loc[df['col_1'].isin(['value_a', 'value_b', 'value_c'])]
```


Logical Operator Precedence

- By default, `&` (AND) operations are executed before `|` (OR) operations.
- We can use parentheses to enforce a particular order of execution.

```
df.loc[(df['col_1'] == True) &  
       ((df['col_2'] == 'Technology') |  
        (df['col_2'] == 'Automotive'))]
```

LESSON 8

Complex Filtering

Top N Filtering

- Select the top N or bottom N rows based on a column value
- In pandas, use `nlargest()` and `smallest()` methods:

```
top_n_df = df.nlargest(n, 'column_name')  
bottom_n_df = df.smallest(n, 'column_name')
```

Intermediate Columns

- Break complex filtering conditions into named boolean columns for clarity and verification
- Create the columns, then use them to filter:

```
df['condition1'] = (df['col_1'] <= 10)
df['condition2'] = (df['col_2'] > 0)
df['condition3'] = (df['col_1'] < df['col_3'])

filtered_df = df.loc[df['condition1'] &
                     (df['condition2'] |
                      df['condition3'])]
```

Direct Calculation in Filtering

- Use expressions directly as conditions in filtering operations:

```
filtered_df = df.loc[  
    (df['col_1'] <= 10) |  
    (df['col_2'] / df['col_3'] > 0.25)]
```

- For simple cases this works well, but for complex conditions, create distinct columns through a dedicated transformation step to keep code clear and reduce mistakes.

Condition Inversion

- Use the `~` (NOT) operator to negate a condition and get the complement:

```
df_complement = df.loc[
    ~((df['col_1'] <= 10) |
      (df['col_2'] / df['col_3'] > 0.25))]
```

- Always enclose compound conditions in parentheses when using `~`.

Result Verification

- Complex filtering conditions are prone to errors—systematic verification catches mistakes before they lead to incorrect conclusions
- Verification happens at two stages:
 1. **While building** — Verify each intermediate column works correctly on its own
 2. **After building** — Verify the final filtered result includes the right rows and excludes the right rows
- Pick a few rows and check them by hand to build confidence that your filter is working correctly

LESSON 9

Conditional Transformation

Conditional Transformation

- Creates new columns by evaluating conditions for each row and applying different logic based on which conditions are met
- Single condition: binary choice between two outcomes
- Multiple conditions: evaluated in order, first match wins

np.where() Function

- Use for single condition with two outcomes
- Syntax: `np.where(condition, value_if_true, value_if_false)`
- Simple binary choice — condition determines which of two values each row receives

np.select() Function

- Use for multiple conditions
- Syntax: `np.select(conditions, outcomes, default)`
- Conditions and outcomes are lists; first True condition wins
- Default value used when no condition matches (returns 0 if not specified)

Binning

- Groups numerical values into discrete categories based on thresholds
- Unlocks data summaries and insights that wouldn't otherwise be possible

LESSON 10

Conditional Aggregation

Conditional Aggregation

- Includes only values meeting specific conditions in calculations
- Same design framework as regular aggregation, with filter condition added

Custom Functions

- Define reusable functions for complex logic
- Pass function name to `agg()`

```
def avg_delayed(values):  
    return values[values > 5].mean()  
  
df.groupby('airline').agg(avg_delay=('delay', avg_delayed))
```

Lambda with agg()

- Use for single-column conditional aggregation
- Filter and aggregate same column

```
df.groupby('airline').agg(  
    avg_delay=('delay', lambda x: x[x > 5].mean())  
)
```


Lambda with apply()

- Use when filtering by one column, aggregating another
- Provides access to entire grouped DataFrame

```
df.groupby('airline').apply(  
    lambda x: pd.Series({  
        'arrived_passengers': x.loc[x['flight_status'] == 'arrived', 'passengers'].sum()  
    }), include_groups=False  
)
```